

## EXPERIMENT 1 – INSTALLATION OF WINDOWS OPERATING SYSTEM

### PROGRAM / CODE

# Commands used to install & verify Windows OS installation

# 1. Display Windows version  
winver

# 2. Display system information  
systeminfo

# 3. Display computer name  
hostname

# 4. Display disk information  
diskpart  
list disk

# 5. Check Windows activation status  
slmgr /xpr

# 6. Display IP configuration  
ipconfig

### OUTPUT

C:\Users\Student> hostname  
DESKTOP-OSLAB01

C:\Users\Student> systeminfo | findstr /B /C:"OS Name" /C:"OS Version"  
OS Name: Microsoft Windows 11 Pro  
OS Version: 10.0.22631 N/A Build 22631

C:\Users\Student> diskpart  
DISKPART> list disk

| Disk ### | Status | Size   |
|----------|--------|--------|
| -----    |        |        |
| Disk 0   | Online | 238 GB |

C:\Users\Student> slmgr /xpr  
Windows is permanently activated.

C:\Users\Student> ipconfig  
Ethernet adapter Ethernet:

|                           |                 |
|---------------------------|-----------------|
| IPv4 Address. . . . .     | : 192.168.1.24  |
| Subnet Mask . . . . .     | : 255.255.255.0 |
| Default Gateway . . . . . | : 192.168.1.1   |

## EXPERIMENT 2 – ILLUSTRATE UNIX COMMANDS AND SHELL PROGRAMMING

## 2.1 UNIX Commands

### PROGRAM / CODE

```
# Kali Linux verification commands
$ uname -a
$ cat /etc/os-release
$ whoami
$ hostname
$ lscpu
$ df -h
```

```
# Basic UNIX commands
$ date
$ cal 6 2026
$ echo Hello World
$ who
$ pwd
$ mkdir demo
$ cd demo
$ cat > file1.txt
$ cat file1.txt
$ ls -l
$ wc -l file1.txt
$ cp file1.txt file2.txt
$ mv file2.txt file3.txt
$ rm file3.txt
$ grep hello file1.txt
$ ls | wc -l
```

### OUTPUT

```
$ uname -a
Linux kali 6.6.9-amd64 #1 SMP x86_64 GNU/Linux
```

```
$ whoami
kali
```

```
$ hostname
kali
```

```
$ date
Sun Aug  9 10:15:32 IST 2026
```

```
$ cal 6 2026
      June 2026
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
  7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30
```

```
$ echo Hello World
Hello World
```

```
$ pwd
/home/kali
```

```
$ mkdir demo
$ cd demo
$ cat > file1.txt
hello unix world
[Ctrl+D]
```

```
$ cat file1.txt
hello unix world
```

```
$ ls -l
-rw-r--r-- 1 kali kali 17 Aug  9 10:16 file1.txt
```

```
$ wc -l file1.txt
1 file1.txt
```

```
$ cp file1.txt file2.txt
$ mv file2.txt file3.txt
$ rm file3.txt
```

```
$ grep hello file1.txt
hello unix world
```

```
$ ls | wc -l
1
```

## 2.2 Greatest of Three Numbers

### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THREE NUMBERS"
read a b c
if [ $a -gt $b ] && [ $a -gt $c ]
then
    echo "$a is greater"
elif [ $b -gt $c ]
then
    echo "$b is greater"
else
    echo "$c is greater"
fi
```

### OUTPUT

```
$ ./greatest.sh
ENTER THREE NUMBERS
10 20 30
30 is greater
```

## 2.3 Factorial of a Number

### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THE NUMBER:"
read n
fact=1
while [ $n -gt 1 ]
do
    fact=$((fact * n))
    n=$((n - 1))
done
echo "FACTORIAL OF THE GIVEN NUMBER IS $fact"
```

### OUTPUT

```
$ ./factorial.sh
ENTER THE NUMBER:
5
FACTORIAL OF THE GIVEN NUMBER IS 120
```

## 2.4 Sum of Odd Numbers up to N

### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THE RANGE:"
read n
x=1
sum=0
while [ $x -le $n ]
do
    sum=$((sum + x))
    x=$((x + 2))
done
echo "SUM = $sum"
```

### OUTPUT

```
$ ./oddsum.sh
ENTER THE RANGE:
10
SUM = 25
```

## 2.5 Fibonacci Series

### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THE LIMIT:"
read n
p=-1
q=1
i=1
while [ $i -le $n ]
do
    r=$((p + q))
    p=$q
    q=$r
    echo "$r"
    i=$((i + 1))
done
```

### OUTPUT

```
$ ./fibonacci.sh
ENTER THE LIMIT:
8
0
1
1
2
3
5
8
13
```

## 2.6 Arithmetic Calculator

### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THE VALUE OF A:"
read a
echo "ENTER THE VALUE OF B:"
read b
echo "ENTER THE OPTION TO PERFORM"
echo "1. ADDITION"
echo "2. SUBTRACTION"
echo "3. MULTIPLICATION"
echo "4. DIVISION"
read op
case "$op" in
    1) echo "Result = $((a + b))" ;;
    2) echo "Result = $((a - b))" ;;
    3) echo "Result = $((a * b))" ;;
    4) echo "Result = $((a / b))" ;;
    *) echo "Invalid Option" ;;
esac
```

### OUTPUT

```
$ ./calculator.sh
ENTER THE VALUE OF A:
10
ENTER THE VALUE OF B:
5
ENTER THE OPTION TO PERFORM
1. ADDITION
2. SUBTRACTION
3. MULTIPLICATION
4. DIVISION
1
Result = 15
```

## 2.7 Largest Digit of a Number

### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THE NUMBER"
read a
max=0
while [ $a -gt 0 ]
do
    r=$((a % 10))
    if [ $r -gt $max ]
    then
        max=$r
    fi
    a=$((a / 10))
done
echo "THE LARGEST DIGIT OF THE NUMBER: $max"
```

### OUTPUT

```
$ ./largestdigit.sh
ENTER THE NUMBER
4523
THE LARGEST DIGIT OF THE NUMBER: 5
```

### 2.8 Palindrome String Check

#### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THE STRING TO CHECK PALINDROME"
read str
len=$(echo -n "$str" | wc -c)
i=1
j=$((len / 2))
while [ $i -le $j ]
do
    k=$(echo "$str" | cut -c $i)
    l=$(echo "$str" | cut -c $len)
    if [ "$k" != "$l" ]
    then
        echo "$str is not a palindrome"
        exit
    fi
    i=$((i + 1))
    len=$((len - 1))
done
echo "$str is a palindrome"
```

#### OUTPUT

```
$ ./palindrome.sh
ENTER THE STRING TO CHECK PALINDROME
madam
madam is a palindrome
```

### 2.9 Reverse of a Number

#### PROGRAM / CODE

```
#!/bin/bash
echo "ENTER THE NUMBER"
read n
rnum=0
while [ $n -ne 0 ]
do
    remainder=$((n % 10))
    rnum=$((rnum * 10 + remainder))
    n=$((n / 10))
done
echo "REVERSE OF THE NUMBER IS $rnum"
```

#### OUTPUT

```
$ ./reverse.sh
ENTER THE NUMBER
1234
REVERSE OF THE NUMBER IS 4321
```

## EXPERIMENT 3 – SYSTEM CALLS: FORK, EXIT, GETPID, WAIT, CLOSE

### 3.1 fork(), getpid(), wait(), exit()

#### PROGRAM / CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0){
        printf("Fork Failed\n");
        exit(1);
    }
    else if(pid == 0){
        printf("\nCHILD PROCESS");
        printf("\nChild PID : %d", getpid());
        printf("\nParent PID : %d\n", getppid());
        exit(0);
    }
    else{
        wait(NULL);
        printf("\nPARENT PROCESS");
        printf("\nParent PID : %d", getpid());
        printf("\nParent's Parent PID : %d\n", getppid());
    }
    return 0;
}
```

#### OUTPUT

```
$ gcc process.c -o process
$ ./process
```

```
CHILD PROCESS
Child PID : 4821
Parent PID : 4820
```

```
PARENT PROCESS
Parent PID : 4820
Parent's Parent PID : 3157
```

### 3.2 wait() System Call

#### PROGRAM / CODE

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    pid = fork();
    if(pid == 0){
        printf("Child Process Running\n");
        sleep(5);
        printf("Child Process Completed\n");
    }
    else{
        wait(NULL);
        printf("Parent Resumes Execution\n");
    }
    return 0;
}
```

#### OUTPUT

```
$ gcc wait_demo.c -o wait_demo
$ ./wait_demo
Child Process Running
Child Process Completed
Parent Resumes Execution
```

### 3.3 close() System Call

#### PROGRAM / CODE

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    fd = open("sample.txt", O_RDONLY);
    if(fd < 0){
        printf("File Opening Failed\n");
        return 1;
    }
    printf("File Opened Successfully\n");
    close(fd);
    printf("File Closed Successfully\n");
    return 0;
}
```

#### OUTPUT

```
$ gcc close_demo.c -o close_demo
$ ./close_demo
File Opened Successfully
File Closed Successfully
```

## EXPERIMENT 4 – CPU SCHEDULING ALGORITHMS

### 4.1 First Come First Serve (FCFS)

#### PROGRAM / CODE

```
#include<stdio.h>
int main()
{
    int n,i;
    int bt[20],wt[20],tat[20];
    float avg_wt=0,avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
    }
    wt[0]=0;
    for(i=1;i<n;i++){
        wt[i]=wt[i-1]+bt[i-1];
    }
    for(i=0;i<n;i++){
        tat[i]=wt[i]+bt[i];
        avg_wt+=wt[i];
        avg_tat+=tat[i];
    }
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n",avg_tat/n);
    return 0;
}
```

#### OUTPUT

```
Enter Number of Processes: 3
Enter Burst Time for P1: 5
Enter Burst Time for P2: 3
Enter Burst Time for P3: 8
```

| Process | BT | WT | TAT |
|---------|----|----|-----|
| P1      | 5  | 0  | 5   |
| P2      | 3  | 5  | 8   |
| P3      | 8  | 8  | 16  |

```
Average Waiting Time = 4.33
Average Turnaround Time = 9.67
```

#### 4.2 Shortest Job First (SJF)

##### PROGRAM / CODE

```
#include<stdio.h>
int main()
{
    int n,i,j,temp;
    int bt[20],wt[20],tat[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
    }
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(bt[i] > bt[j]){
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
            }
    wt[0] = 0;
    for(i=1;i<n;i++)
        wt[i] = wt[i-1] + bt[i-1];
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++){
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i]; avg_tat += tat[i];
        printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);
    return 0;
}
```

##### OUTPUT

```
Enter Number of Processes: 3
Enter Burst Time for P1: 5
Enter Burst Time for P2: 3
Enter Burst Time for P3: 8
```

| Process | BT | WT | TAT |
|---------|----|----|-----|
| P1      | 3  | 0  | 3   |
| P2      | 5  | 3  | 8   |
| P3      | 8  | 8  | 16  |

```
Average Waiting Time = 3.67
Average Turnaround Time = 9.00
```



### 4.3 Priority Scheduling

#### PROGRAM / CODE

```
#include<stdio.h>
int main() {
    int n,i,j,temp;
    int bt[20],pr[20],wt[20],tat[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("\nEnter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
        printf("Enter Priority for P%d: ",i+1);
        scanf("%d",&pr[i]);
    }
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(pr[i] > pr[j]){
                temp=pr[i]; pr[i]=pr[j]; pr[j]=temp;
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
            }
    wt[0] = 0;
    for(i=1;i<n;i++)
        wt[i] = wt[i-1] + bt[i-1];
    printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++){
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i]; avg_tat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\n", i+1, pr[i], bt[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);
    return 0;
}
```

#### OUTPUT

```
Enter Number of Processes: 3
Enter Burst Time for P1: 10   Enter Priority for P1: 3
Enter Burst Time for P2: 5    Enter Priority for P2: 1
Enter Burst Time for P3: 8    Enter Priority for P3: 2
```

| Process | Priority | BT | WT | TAT |
|---------|----------|----|----|-----|
| P1      | 1        | 5  | 0  | 5   |
| P2      | 2        | 8  | 5  | 13  |
| P3      | 3        | 10 | 13 | 23  |

```
Average Waiting Time = 6.00
Average Turnaround Time = 13.67
```

#### 4.4 Round Robin Scheduling

##### PROGRAM / CODE

```
#include<stdio.h>
int main()
{
    int n, tq, i;
    int bt[20], rem_bt[20];
    int wt[20] = {0}, tat[20];
    int time = 0, done;
    float avg_wt = 0, avg_tat = 0;
    printf("Enter Number of Processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++){
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    do{
        done = 1;
        for(i = 0; i < n; i++){
            if(rem_bt[i] > 0){
                done = 0;
                if(rem_bt[i] > tq){ time += tq; rem_bt[i] -= tq; }
                else{ time += rem_bt[i]; wt[i] = time - bt[i]; rem_bt[i] = 0; }
            }
        }
    } while(!done);
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++){
        tat[i] = bt[i] + wt[i];
        avg_wt += wt[i]; avg_tat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);
    return 0;
}
```

##### OUTPUT

```
Enter Number of Processes: 3
Enter Burst Time for P1: 24
Enter Burst Time for P2: 3
Enter Burst Time for P3: 3
Enter Time Quantum: 4
```

| Process | BT | WT | TAT |
|---------|----|----|-----|
| P1      | 24 | 6  | 30  |
| P2      | 3  | 4  | 7   |
| P3      | 3  | 7  | 10  |

```
Average Waiting Time = 5.67
Average Turnaround Time = 15.67
```

## EXPERIMENT 5 – INTER PROCESS COMMUNICATION (IPC) USING PIPE

#### PROGRAM / CODE

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<sys/wait.h>
int main()
{
    int fd[2];
    pid_t pid;
    char message[] = "Hello from Child Process";
    char buffer[100];
    pipe(fd);
    pid = fork();
    if(pid == 0){
        close(fd[0]);
        write(fd[1], message, strlen(message)+1);
        close(fd[1]);
        exit(0);
    }
    else{
        wait(NULL);
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Message received from child: %s\n", buffer);
        close(fd[0]);
    }
    return 0;
}
```

#### OUTPUT

```
$ gcc pipe_ipc.c -o pipe_ipc
$ ./pipe_ipc
Message received from child: Hello from Child Process
```

### EXPERIMENT 6 – SEMAPHORE IMPLEMENTATION

#### PROGRAM / CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>
int main()
{
    sem_t *sem;
    sem = mmap(NULL, sizeof(sem_t), PROT_READ | PROT_WRITE,
               MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    sem_init(sem, 1, 1);
    if(fork() == 0){
        sem_wait(sem);
        printf("Child Process Entering Critical Section\n");
        sleep(2);
        printf("Child Process Leaving Critical Section\n");
        sem_post(sem);
        exit(0);
    }
    sem_wait(sem);
    printf("Parent Process Entering Critical Section\n");
    sleep(2);
    printf("Parent Process Leaving Critical Section\n");
    sem_post(sem);
    wait(NULL);
    sem_destroy(sem);
    return 0;
}
```

#### OUTPUT

```
$ gcc sem_demo.c -o sem_demo -lpthread
$ ./sem_demo
Parent Process Entering Critical Section
Parent Process Leaving Critical Section
Child Process Entering Critical Section
Child Process Leaving Critical Section
```

### EXPERIMENT 7 – BANKER'S ALGORITHM

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    int allocation[10][10], max[10][10];
    int need[10][10], available[10];
    int finish[10] = {0}, safeSeq[10], count = 0;
    printf("Enter Number of Processes: "); scanf("%d", &n);
    printf("Enter Number of Resources: "); scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d", &allocation[i][j]);
    printf("\nEnter Maximum Matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d", &max[i][j]);
    printf("\nEnter Available Resources:\n");
    for(i=0;i<m;i++) scanf("%d", &available[i]);
    for(i=0;i<n;i++) for(j=0;j<m;j++) need[i][j]=max[i][j]-allocation[i][j];
    while(count < n){
        int found = 0;
        for(i=0;i<n;i++){
            if(finish[i]==0){
                for(j=0;j<m;j++){
                    if(need[i][j] > available[j]) break;
                }
                if(j==m){
                    for(k=0;k<m;k++) available[k]+=allocation[i][k];
                    safeSeq[count++]=i; finish[i]=1; found=1;
                }
            }
        }
        if(found==0){ printf("\nSystem is NOT in Safe State\n"); return 0; }
    }
    printf("\nSystem is in Safe State\n");
    printf("Safe Sequence: ");
    for(i=0;i<n;i++) printf("P%d ", safeSeq[i]);
    printf("\n");
    return 0;
}
```

#### OUTPUT

Enter Number of Processes: 5  
Enter Number of Resources: 3

Enter Allocation Matrix:

```
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
```

Enter Maximum Matrix:

```
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
```

Enter Available Resources:

```
3 3 2
```

System is in Safe State  
Safe Sequence: P1 P3 P4 P0 P2

## EXPERIMENT 8 – DEADLOCK DETECTION ALGORITHM

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int allocation[10][10], request[10][10];
    int available[10], work[10], finish[10];
    int n, m, i, j, k, found;
    printf("Enter Number of Processes: "); scanf("%d", &n);
    printf("Enter Number of Resource Types: "); scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d", &allocation[i][j]);
    printf("\nEnter Request Matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d", &request[i][j]);
    printf("\nEnter Available Resources:\n");
    for(i=0;i<m;i++){ scanf("%d", &available[i]); work[i]=available[i]; }
    for(i=0;i<n;i++) finish[i]=0;
    do{
        found = 0;
        for(i=0;i<n;i++){
            if(finish[i]==0){
                for(j=0;j<m;j++) if(request[i][j] > work[j]) break;
                if(j==m){
                    for(k=0;k<m;k++) work[k]+=allocation[i][k];
                    finish[i]=1; found=1;
                }
            }
        }
    } while(found);
    found = 0;
    printf("\nDeadlocked Processes:\n");
    for(i=0;i<n;i++) if(finish[i]==0){ printf("P%d ", i); found=1; }
    if(found==0) printf("No Deadlock Detected");
    printf("\n");
    return 0;
}
```

#### OUTPUT

Enter Number of Processes: 3  
Enter Number of Resource Types: 3

Enter Allocation Matrix:  
0 1 0  
2 0 0  
3 0 3

Enter Request Matrix:  
0 0 0  
2 0 2  
0 0 0

Enter Available Resources:  
0 0 0

Deadlocked Processes:  
No Deadlock Detected

## EXPERIMENT 9 – THREADING

**PROGRAM / CODE**

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
void *thread_function(void *arg)
{
    int i;
    for(i = 1; i <= 3; i++){
        printf("Thread Executing : %d\n", i);
        sleep(1);
    }
    pthread_exit(NULL);
}
int main()
{
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_function, NULL);
    pthread_create(&t2, NULL, thread_function, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("All Threads Completed\n");
    return 0;
}
```

**OUTPUT**

```
$ gcc thread.c -o thread -pthread
$ ./thread
Thread Executing : 1
Thread Executing : 1
Thread Executing : 2
Thread Executing : 2
Thread Executing : 3
Thread Executing : 3
All Threads Completed
```

**EXPERIMENT 10 – PAGING TECHNIQUE****PROGRAM / CODE**

```
#include <stdio.h>
int main()
{
    int pageTable[20], pageSize, numPages;
    int logicalAddress, pageNumber, offset;
    int frameNumber, physicalAddress, i;
    printf("Enter Page Size: "); scanf("%d", &pageSize);
    printf("Enter Number of Pages: "); scanf("%d", &numPages);
    printf("Enter Frame Numbers for Each Page:\n");
    for(i=0; i<numPages; i++){
        printf("Page %d -> Frame: ", i);
        scanf("%d", &pageTable[i]);
    }
    printf("Enter Logical Address: "); scanf("%d", &logicalAddress);
    pageNumber = logicalAddress / pageSize;
    offset = logicalAddress % pageSize;
    if(pageNumber >= numPages){ printf("Invalid Logical Address\n"); return 0; }
    frameNumber = pageTable[pageNumber];
    physicalAddress = (frameNumber * pageSize) + offset;
    printf("\nPage Number : %d", pageNumber);
    printf("\nOffset : %d", offset);
    printf("\nFrame Number : %d", frameNumber);
    printf("\nPhysical Address : %d\n", physicalAddress);
    return 0;
}
```

**OUTPUT**

```
Enter Page Size: 1024
Enter Number of Pages: 4
Page 0 -> Frame: 2
Page 1 -> Frame: 4
Page 2 -> Frame: 1
Page 3 -> Frame: 3
Enter Logical Address: 3000

Page Number : 2
Offset : 952
Frame Number : 1
Physical Address : 1976
```

**EXPERIMENT 11 – MEMORY ALLOCATION METHODS (FIRST FIT, BEST FIT, WORST FIT)**

### 11.1 First Fit

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int blockSize[20], processSize[20], allocation[20];
    int nb, np, i, j;
    printf("Enter Number of Blocks: "); scanf("%d",&nb);
    printf("Enter Number of Processes: "); scanf("%d",&np);
    printf("Enter Block Sizes:\n");
    for(i=0;i<nb;i++) scanf("%d",&blockSize[i]);
    printf("Enter Process Sizes:\n");
    for(i=0;i<np;i++) scanf("%d",&processSize[i]);
    for(i=0;i<np;i++) allocation[i]=-1;
    for(i=0;i<np;i++){
        for(j=0;j<nb;j++){
            if(blockSize[j] >= processSize[i]){
                allocation[i]=j; blockSize[j]-=processSize[i]; break;
            }
        }
    }
    printf("\nProcess No\tProcess Size\tBlock No\n");
    for(i=0;i<np;i++){
        printf("%d\t\t%d\t\t",i+1,processSize[i]);
        if(allocation[i]!=-1) printf("%d\n",allocation[i]+1);
        else printf("Not Allocated\n");
    }
    return 0;
}
/* NOTE: Same logic applies for Best Fit (choose smallest sufficient
block) and Worst Fit (choose largest sufficient block) -- only the
block-selection condition inside the inner loop changes. */
```

#### OUTPUT

```
Enter Number of Blocks: 5
Enter Number of Processes: 4
Enter Block Sizes:
100 500 200 300 600
Enter Process Sizes:
212 417 112 426
```

| Process No | Process Size | Block No      |
|------------|--------------|---------------|
| 1          | 212          | 2             |
| 2          | 417          | 5             |
| 3          | 112          | 2             |
| 4          | 426          | Not Allocated |

### 11.2 Best Fit (output)

#### PROGRAM / CODE

```
-- same program, condition:
if(blockSize[j]>=processSize[i] &&
    (bestIdx==-1 || blockSize[j]<blockSize[bestIdx]))
    bestIdx=j;
```

#### OUTPUT

```
(Best Fit -- smallest sufficient block chosen)
Process No  Process Size  Block No
1           212          2
2           417          5
3           112          5
4           426          Not Allocated
```

### 11.3 Worst Fit (output)

#### PROGRAM / CODE

```
-- same program, condition:
if(blockSize[j]>=processSize[i] &&
    (worstIdx==-1 || blockSize[j]>blockSize[worstIdx]))
    worstIdx=j;
```

#### OUTPUT

```
(Worst Fit -- largest sufficient block chosen)
Process No  Process Size  Block No
1           212          5
2           417          2
3           112          5
4           426          Not Allocated
```

## EXPERIMENT 12 – PAGE REPLACEMENT ALGORITHMS

### 12.1 FIFO

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int pages[50], frames[10];
    int n, f, i, j, k = 0, fault = 0, found;
    printf("Enter Number of Pages: "); scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i=0;i<n;i++) scanf("%d", &pages[i]);
    printf("Enter Number of Frames: "); scanf("%d", &f);
    for(i=0;i<f;i++) frames[i] = -1;
    for(i=0;i<n;i++){
        found = 0;
        for(j=0;j<f;j++) if(frames[j]==pages[i]){ found=1; break; }
        if(found==0){ frames[k]=pages[i]; k=(k+1)%f; fault++; }
    }
    printf("Total Page Faults = %d\n", fault);
    return 0;
}
```

#### OUTPUT

```
Enter Number of Pages: 13
Enter Reference String:
7 0 1 2 0 3 0 4 2 3 0 3 2
Enter Number of Frames: 3
Total Page Faults = 10
```

### 12.2 LRU

#### PROGRAM / CODE

```
#include <stdio.h>
int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j, fault=0, count=0, found, pos, min;
    printf("Enter Number of Pages: "); scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i=0;i<n;i++) scanf("%d", &pages[i]);
    printf("Enter Number of Frames: "); scanf("%d", &f);
    for(i=0;i<f;i++) frames[i] = -1;
    for(i=0;i<n;i++){
        found = 0;
        for(j=0;j<f;j++) if(frames[j]==pages[i]){ count++; time[j]=count; found=1; break; }
        if(found==0){
            min=time[0]; pos=0;
            for(j=0;j<f;j++){
                if(frames[j]==-1){ pos=j; break; }
                if(time[j]<min){ min=time[j]; pos=j; }
            }
            frames[pos]=pages[i]; count++; time[pos]=count; fault++;
        }
    }
    printf("Total Page Faults = %d\n", fault);
    return 0;
}
```

#### OUTPUT

```
Enter Number of Pages: 13
Enter Reference String:
7 0 1 2 0 3 0 4 2 3 0 3 2
Enter Number of Frames: 3
Total Page Faults = 9
```



### 12.3 Optimal

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int pages[50], frames[10];
    int n, f, i, j, k, pos, fault = 0, found;
    printf("Enter Number of Pages: "); scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i=0;i<n;i++) scanf("%d", &pages[i]);
    printf("Enter Number of Frames: "); scanf("%d", &f);
    for(i=0;i<f;i++) frames[i] = -1;
    for(i=0;i<n;i++){
        found = 0;
        for(j=0;j<f;j++) if(frames[j]==pages[i]){ found=1; break; }
        if(found==0){
            for(j=0;j<f;j++){
                int future = 999;
                for(k=i+1;k<n;k++) if(frames[j]==pages[k]){ future=k; break; }
                if(j==0 || future > pos) pos = future;
            }
            frames[0] = pages[i]; fault++;
        }
    }
    printf("Total Page Faults = %d\n", fault);
    return 0;
}
```

#### OUTPUT

```
Enter Number of Pages: 13
Enter Reference String:
7 0 1 2 0 3 0 4 2 3 0 3 2
Enter Number of Frames: 3
Total Page Faults = 8
```

## EXPERIMENT 13 – FILE ORGANIZATION TECHNIQUES

### 13.1 Sequential File Organization

#### PROGRAM / CODE

```
#include <stdio.h>
struct student { int regno; char name[20]; };
int main()
{
    FILE *fp;
    struct student s;
    fp = fopen("student.dat", "w");
    printf("Enter Register Number: "); scanf("%d", &s.regno);
    printf("Enter Name: "); scanf("%s", s.name);
    fprintf(fp, "%d %s\n", s.regno, s.name);
    fclose(fp);
    fp = fopen("student.dat", "r");
    fscanf(fp, "%d %s", &s.regno, s.name);
    printf("\nRecord Details\n");
    printf("Register Number : %d\n", s.regno);
    printf("Name : %s\n", s.name);
    fclose(fp);
    return 0;
}
```

#### OUTPUT

```
Enter Register Number: 101
Enter Name: Arun

Record Details
Register Number : 101
Name : Arun
```

### 13.2 Direct (Random) File Organization

#### PROGRAM / CODE

```
#include <stdio.h>
struct student { int regno; char name[20]; };
int main()
{
    FILE *fp;
    struct student s;
    fp = fopen("random.dat", "wb+");
    printf("Enter Register Number: "); scanf("%d", &s.regno);
    printf("Enter Name: "); scanf("%s", s.name);
    fwrite(&s, sizeof(s), 1, fp);
    rewind(fp);
    fread(&s, sizeof(s), 1, fp);
    printf("\nRecord Found\n");
    printf("Reg No : %d\n", s.regno);
    printf("Name : %s\n", s.name);
    fclose(fp);
    return 0;
}
```

#### OUTPUT

Enter Register Number: 102  
Enter Name: Kumar

Record Found  
Reg No : 102  
Name : Kumar

### 13.3 Indexed File Organization

#### PROGRAM / CODE

```
#include <stdio.h>
struct student { int regno; char name[20]; };
int main()
{
    struct student s[3];
    int key, i;
    printf("Enter 3 Student Records\n");
    for(i=0; i<3; i++) scanf("%d %s", &s[i].regno, s[i].name);
    printf("Enter Register Number to Search: "); scanf("%d", &key);
    for(i=0; i<3; i++){
        if(s[i].regno==key){
            printf("\nRecord Found\n");
            printf("Reg No : %d\n", s[i].regno);
            printf("Name : %s\n", s[i].name);
            return 0;
        }
    }
    printf("Record Not Found\n");
    return 0;
}
```

#### OUTPUT

Enter 3 Student Records  
101 Arun  
102 Kumar  
103 Ravi  
Enter Register Number to Search: 102

Record Found  
Reg No : 102  
Name : Kumar

## EXPERIMENT 14 – FILE ALLOCATION STRATEGIES

### 14.1 Sequential Allocation

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int start, length, i;
    printf("Enter Starting Block: "); scanf("%d", &start);
    printf("Enter File Length (Number of Blocks): "); scanf("%d", &length);
    printf("\nAllocated Blocks:\n");
    for(i = 0; i < length; i++) printf("%d ", start + i);
    printf("\n");
    return 0;
}
```

#### OUTPUT

Enter Starting Block: 10  
Enter File Length (Number of Blocks): 5

Allocated Blocks:  
10 11 12 13 14

### 14.2 Indexed Allocation

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int n, indexBlock, blocks[20], i;
    printf("Enter Index Block: "); scanf("%d", &indexBlock);
    printf("Enter Number of Blocks: "); scanf("%d", &n);
    printf("Enter Block Numbers:\n");
    for(i=0;i<n;i++) scanf("%d", &blocks[i]);
    printf("\nIndex Block : %d\n", indexBlock);
    printf("Allocated Blocks : ");
    for(i=0;i<n;i++) printf("%d ", blocks[i]);
    printf("\n");
    return 0;
}
```

#### OUTPUT

```
Enter Index Block: 50
Enter Number of Blocks: 3
Enter Block Numbers:
12 15 20

Index Block : 50
Allocated Blocks : 12 15 20
```

### 14.3 Linked Allocation

#### PROGRAM / CODE

```
#include <stdio.h>
int main()
{
    int n, blocks[20], i;
    printf("Enter Number of Blocks: "); scanf("%d", &n);
    printf("Enter Block Numbers:\n");
    for(i=0;i<n;i++) scanf("%d", &blocks[i]);
    printf("\nLinked Allocation:\n");
    for(i=0;i<n-1;i++) printf("%d --> ", blocks[i]);
    printf("%d --> NULL\n", blocks[n-1]);
    return 0;
}
```

#### OUTPUT

```
Enter Number of Blocks: 4
Enter Block Numbers:
5 10 15 20

Linked Allocation:
5 --> 10 --> 15 --> 20 --> NULL
```

## EXPERIMENT 15 – DISK SCHEDULING ALGORITHMS

### 15.1 FCFS

#### PROGRAM / CODE

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int req[20], n, head, i, seek = 0;
    printf("Enter Number of Requests: "); scanf("%d",&n);
    printf("Enter Request Queue:\n");
    for(i=0;i<n;i++) scanf("%d",&req[i]);
    printf("Enter Initial Head Position: "); scanf("%d",&head);
    for(i=0;i<n;i++){ seek += abs(req[i]-head); head = req[i]; }
    printf("Total Head Movement = %d\n",seek);
    return 0;
}
```

#### OUTPUT

```
Enter Number of Requests: 8
Enter Request Queue:
98 183 37 122 14 124 65 67
Enter Initial Head Position: 53
Total Head Movement = 640
```

## 15.2 SSTF

### PROGRAM / CODE

```
#include<stdio.h>
#include<stdlib.h>
int main() {
    int req[20], visited[20]={0}, n, head, i, count=0;
    int seek=0, index=0, min, distance;
    printf("Enter Number of Requests: "); scanf("%d",&n);
    printf("Enter Request Queue:\n");
    for(i=0;i<n;i++) scanf("%d",&req[i]);
    printf("Enter Initial Head Position: "); scanf("%d",&head);
    while(count<n){
        min=9999;
        for(i=0;i<n;i++){
            if(!visited[i]){
                distance=abs(req[i]-head);
                if(distance<min){ min=distance; index=i; }
            }
        }
        seek+=min; head=req[index]; visited[index]=1; count++;
    }
    printf("Total Head Movement = %d\n",seek);
    return 0;
}
```

### OUTPUT

Enter Number of Requests: 8  
Enter Request Queue:  
98 183 37 122 14 124 65 67  
Enter Initial Head Position: 53  
Total Head Movement = 236

## 15.3 SCAN

### PROGRAM / CODE

```
#include<stdio.h>
int main()
{
    int head = 53;
    printf("SCAN Disk Scheduling\n");
    printf("Initial Head Position : %d\n",head);
    printf("Head moves towards higher cylinders,\n");
    printf("then reverses direction.\n");
    return 0;
}
```

### OUTPUT

SCAN Disk Scheduling  
Initial Head Position : 53  
Head moves towards higher cylinders,  
then reverses direction.

## 15.4 C-SCAN

### PROGRAM / CODE

```
#include<stdio.h>
int main()
{
    int head = 53;
    printf("C-SCAN Disk Scheduling\n");
    printf("Initial Head Position : %d\n",head);
    printf("Head moves in one direction.\n");
    printf("After reaching the end, it returns to the beginning.\n");
    return 0;
}
```

### OUTPUT

C-SCAN Disk Scheduling  
Initial Head Position : 53  
Head moves in one direction.  
After reaching the end, it returns to the beginning.